

INSO & MAIS



# Procesamiento de datos

Miguel Ángel Fernández Díaz  
2025-2026



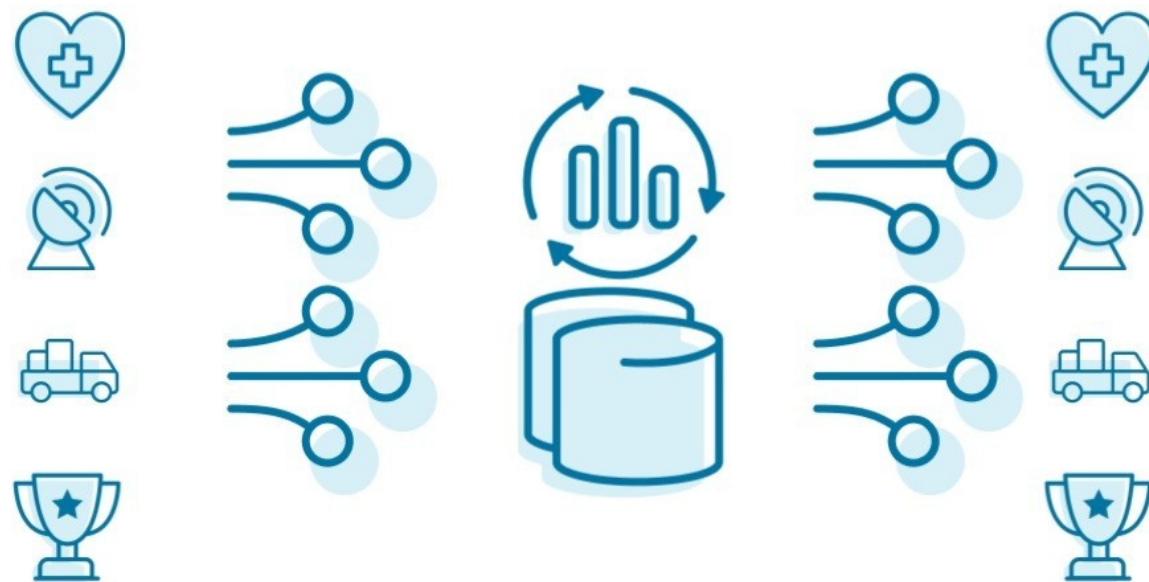
# **Tema 5:**

## Spark Streaming

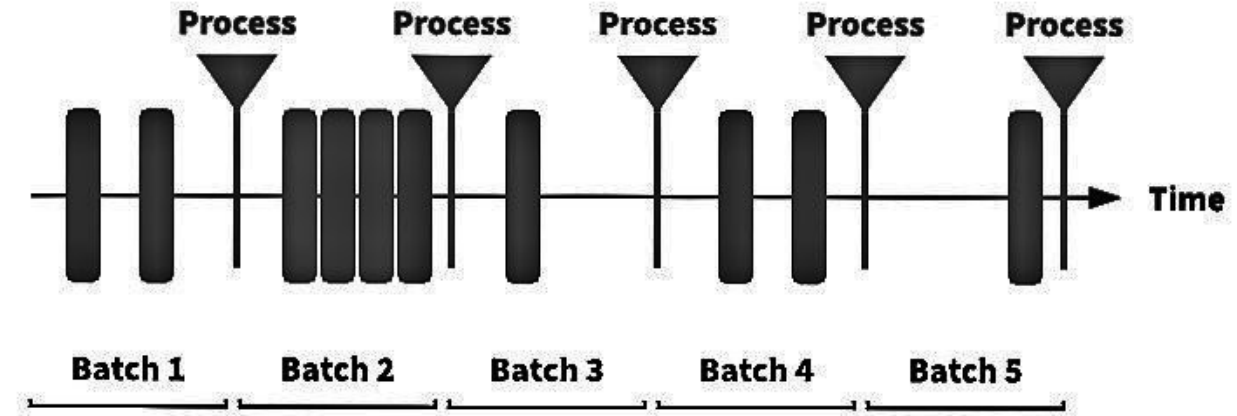
1. Introducción a Sistemas Streaming
2. Apache Kafka
3. Spark Streaming: Structured Streaming

# **1. Introducción a Sistemas Streaming**

- Características:
  - El sistema que queremos monitorizar equivale a una corriente continua e infinita de eventos o de datos
  - El flujo tiene un inicio pero no un final
  - No pueden tratarse todos los eventos a la vez, como en batch, ya que son potencialmente infinitos. Deben usarse ventanas temporales o por número de eventos
  - Respuesta (casi) inmediata
  - Algunos casos de uso típicos: IoT, fraude, ciberseguridad, dashboard en tiempo real, monitorización en tiempo real, etc.
  - Tecnologías: Flink, Kafka, Spark Streaming, Storm, Kinesis, etc.



- Real-Time:
  - Procesamiento continuo
  - Latencia baja
  - Uso más intensivo de los recursos del entorno
  - Diseño complejo para escalar
  - Lógica específica distinta de un Batch “clásico”
- Micro-batch o Near-Real-Time
  - Procesamiento por lotes pequeños
  - Latencia relativamente alta
  - Uso discontinuo de los recursos del entorno
  - Diseño simple para escalar
  - Lógica de transformación igual que un Batch “clásico”

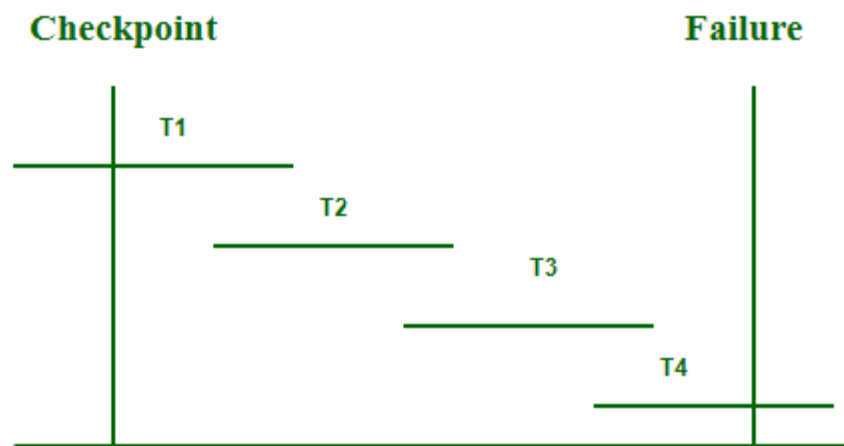


- Replicación de Datos:

- Estrategia fundamental para la **tolerancia a fallos** en sistemas de streaming. Los sistemas pueden replicar los datos de streaming en múltiples nodos o particiones para garantizar la disponibilidad de los datos en caso de que un **nodo o partición** falle.

- Checkpointing

- Técnica que guarda periódicamente el estado del procesamiento de streaming en un almacenamiento duradero, como HDFS o Amazon S3. Si se produce un fallo, el sistema puede reiniciar desde el último checkpoint guardado para recuperar el estado del procesamiento y continuar desde ese punto.



- Microbatches
  - Procesa elementos en base a un delay. Los agrupa.
  - Hasta Spark 2.3 era el comportamiento estándar
    - 100 ms latencia
- Real time
  - Procesa elementos sin delay (microsegundos)
  - Reacciona
  - Spark 2.3
    - [Continuous processing Engine](#)
    - 1ms latencia
- Event time
  - Momento en el que se **generó** el evento
- Ingestion time
  - Momento en el que el evento **llegó** al sistema
- Processing time
  - Momento en el que es **procesado**

- at-least-once
  - Asegura que cada evento se procesa
  - No puede asegurar que no haya duplicados
- at-most-once
  - Asegura que no hay duplicados
  - Pueden perderse eventos
- exactly-once
  - Cada evento se procesa una única vez
  - No hay duplicados
  - No hay pérdidas
  - Spark trabaja con esta semántica



**Mathias Verraes**  
@mathiasverraes

 Follow

There are only two hard problems in distributed systems: 2. Exactly-once delivery 1. Guaranteed order of messages 2. Exactly-once delivery

RETWEETS  
**6,775**

LIKES  
**4,727**



10:40 AM - 14 Aug 2015

 69

 6.8K

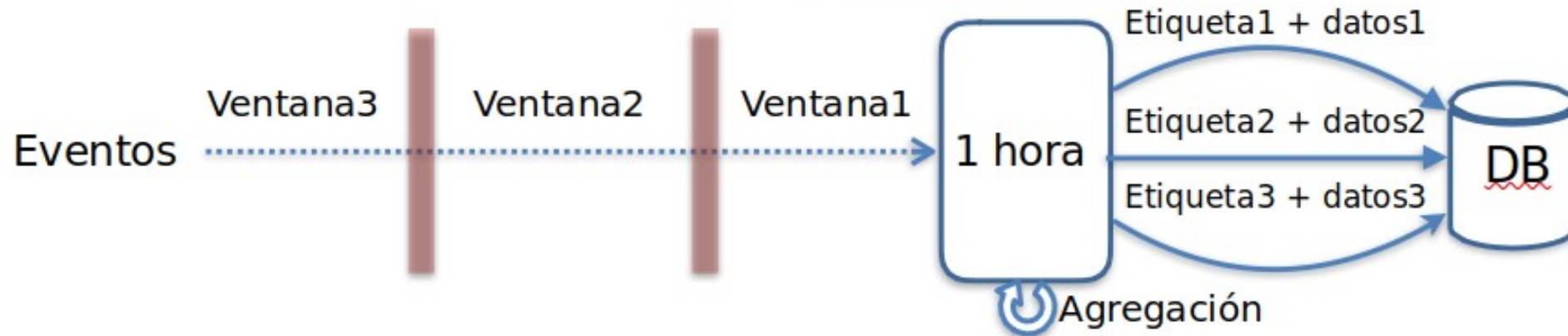
 4.7K



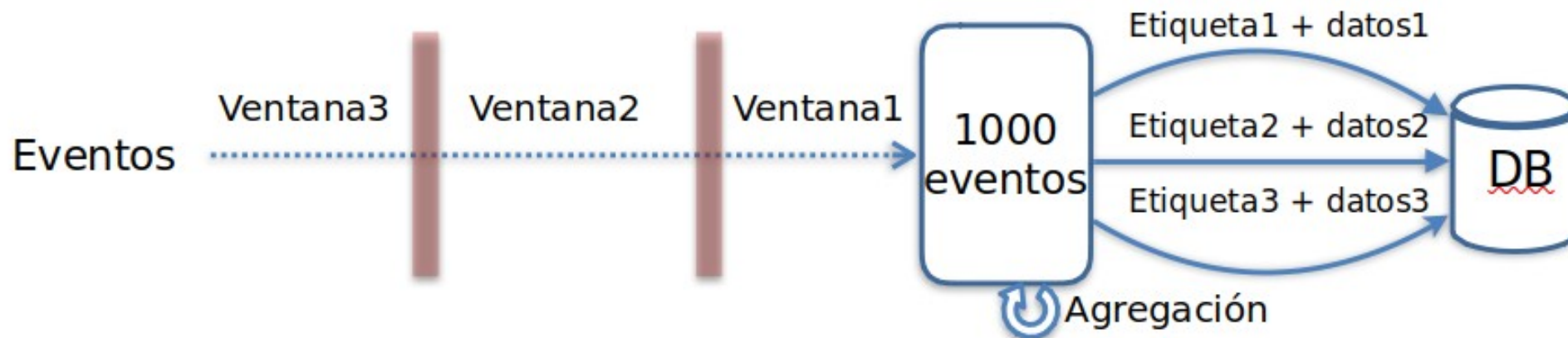
## ● Ventanas

- Temporal (Tumbling Window):
  - Divide el flujo de datos en intervalos de tiempo fijos y discretos
  - Ejemplo: Calculando el promedio de ventas por hora en un sistema de comercio electrónico.
- Deslizante (Sliding Window):
  - Divide el flujo de datos en intervalos de tiempo superpuestos
  - Ejemplo: Calculando la suma de ventas en los últimos 10 minutos, actualizándola cada minuto.
- De Sesión (Session Window):
  - Divide el flujo de datos en sesiones de actividad basadas en ciertos criterios, como la inactividad durante un período de tiempo específico
  - Ejemplo: Analizando el comportamiento del usuario en un sitio web y segmentándolo en sesiones basadas en la inactividad entre páginas.
- Ventana Basada en Conteo (Count Window):
  - Divide el flujo de datos en intervalos de tamaño fijo basados en el número de eventos recibidos.
  - Ejemplo: Realizando un análisis de sentimientos sobre los últimos 100 tweets recibidos.

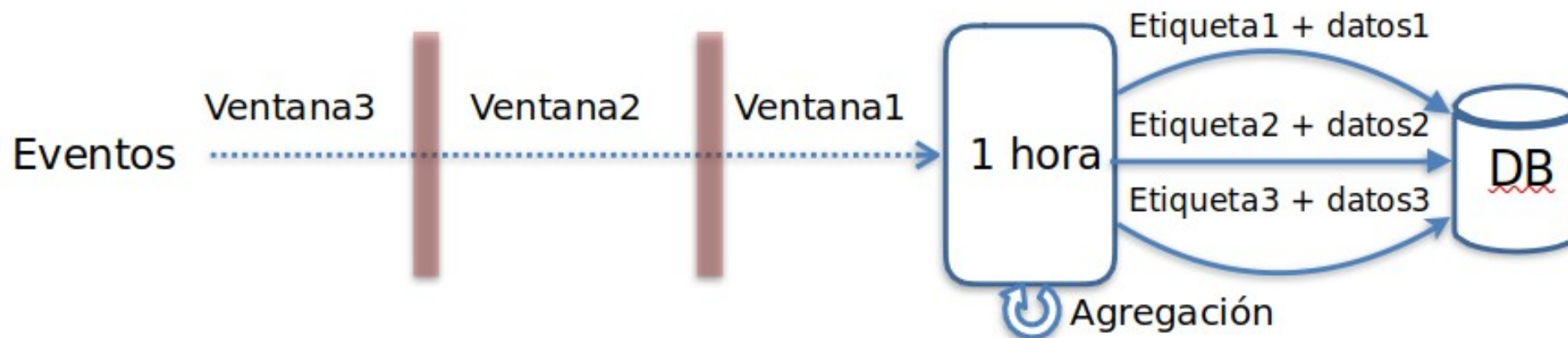
## Ventana temporal



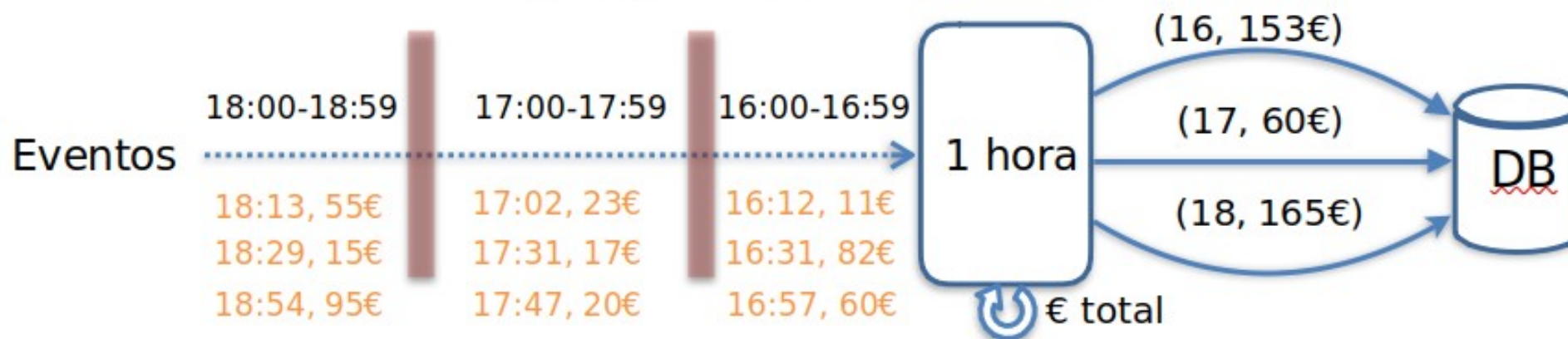
## Ventana de eventos



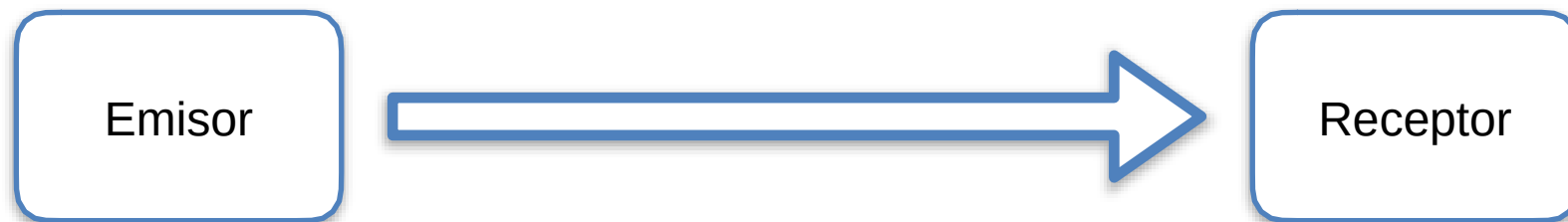
## Ventana temporal



## Ejemplo de ventas en una web



- Backpressure
  - Proceso Streaming: un emisor envía datos a un receptor de forma continua
  - El emisor puede emitir eventos a una mayor velocidad a lo que el receptor puede procesar, provocándole una saturación o pérdida de datos
  - Solución: el receptor le informa al emisor de su capacidad para que reduzca el ritmo de emisión



Ritmo:  
1000 eventos / seg

Capacidad:  
500 eventos / seg

- Transformaciones dependen únicamente de un evento
- No importa el orden
- No almacenan estado
- Pueden recuperarse en cualquier momento
- Tan sólo requieren buffer de elementos no procesados

Transformation applied on single stream entity

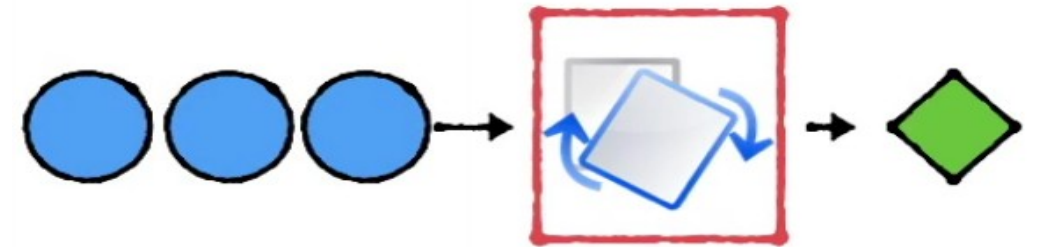


Stateless  
Transformation

- Las transformaciones requieren un subconjunto de elementos para calcularse
- El orden puede ser relevante
- Almacena estado
- Necesita almacenar estado actual por si hay que recuperarse
- El resultado se va acumulando

- Pueden calcularse en base a diferentes agrupaciones:

- Stream completo
- Subconjunto (Ventana)



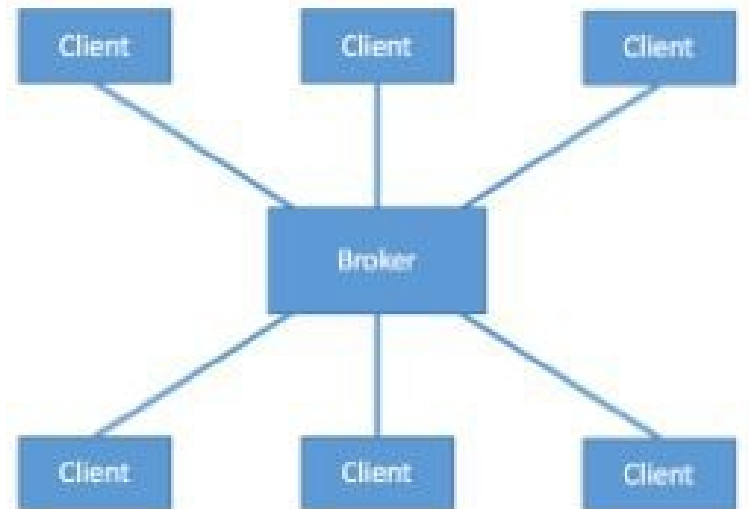
Stateful  
Transformation

## 2. Apache Kafka

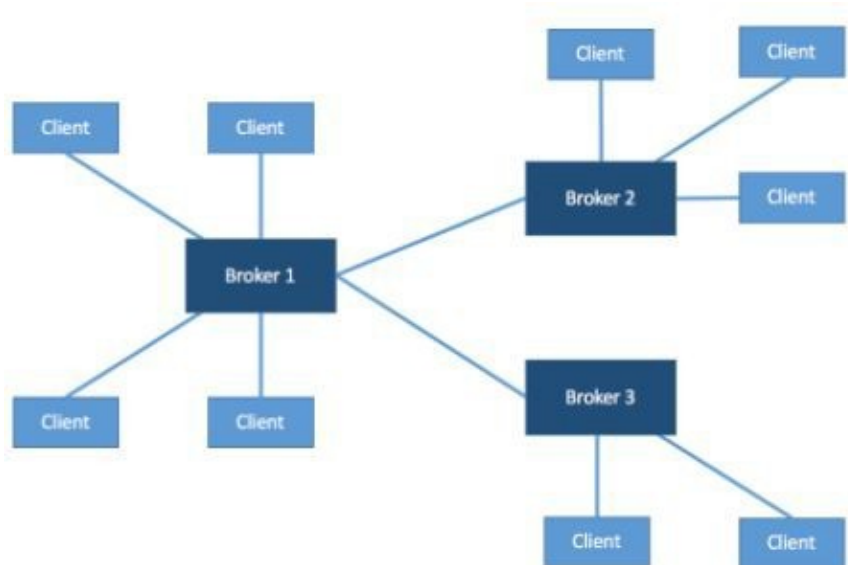
- <http://kafka.apache.org/>
- Creado en LinkedIn, open sourced en el año 2011
- Implementado en Scala (principalmente) y en Java
- 50 core committers, cientos de contributors
- Los desarrolladores de Apache Kafka fundaron en el año 2014 la startup Confluent
- <http://www.confluent.io/training>
- Motivación: plataforma unificada para el manejo de todos los datos en tiempo real de una gran empresa
- Características:
  - Alto rendimiento
  - Durabilidad
  - Alta disponibilidad
  - Escalabilidad



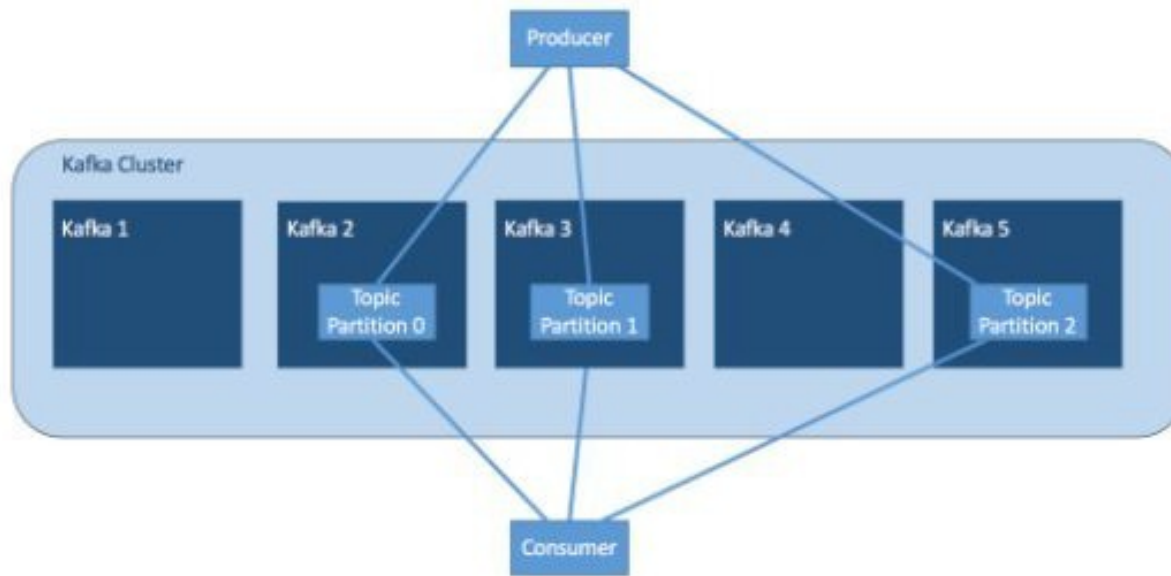
- Broker:
  - Intermediario entre los productores de datos y los consumidores de datos
  - Responsable de recibir, almacenar temporalmente y enrutar los eventos de datos desde los productores hacia los consumidores
- Sistemas tradicionales:
  - El productor envía los mensajes al broker
  - El broker los guarda (memoria o disco)
  - El cliente los consume
  - El broker borra el mensaie



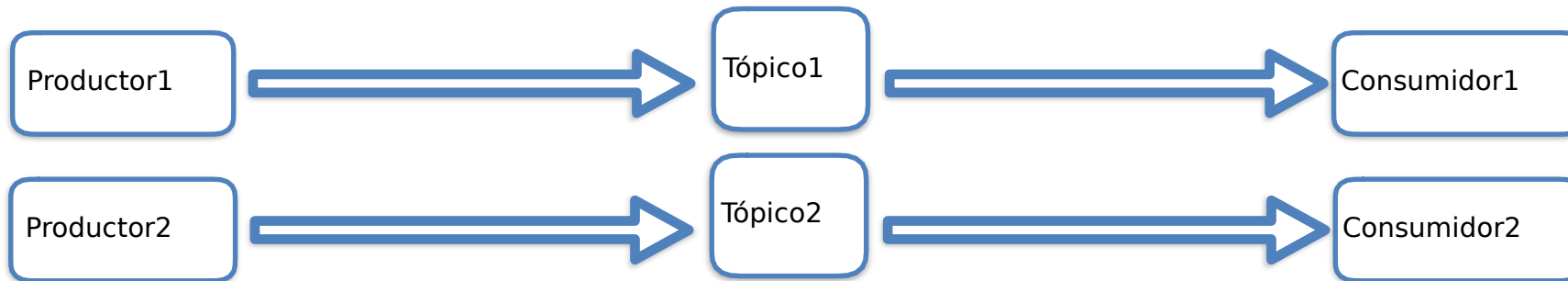
- Sistemas tradicionales:
  - Los brokers se dividen los tópicos
  - Los clientes se conectan solamente al broker que contiene su tópico
  - No escala horizontalmente



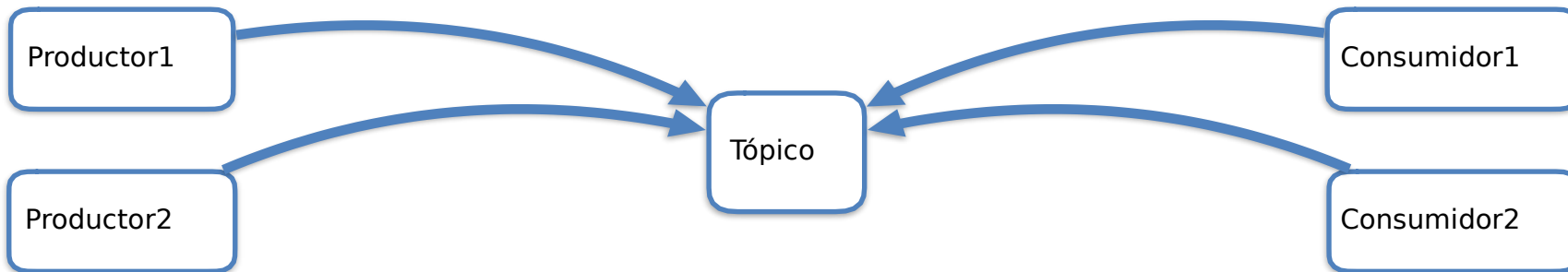
- Propuesta de Kafka:
  - Tópico dividido en diferentes brokers y replicado
  - Escalado horizontal + vertical
  - Todos los mensajes son persistidos a disco, no se borran tras la entrega al consumidor



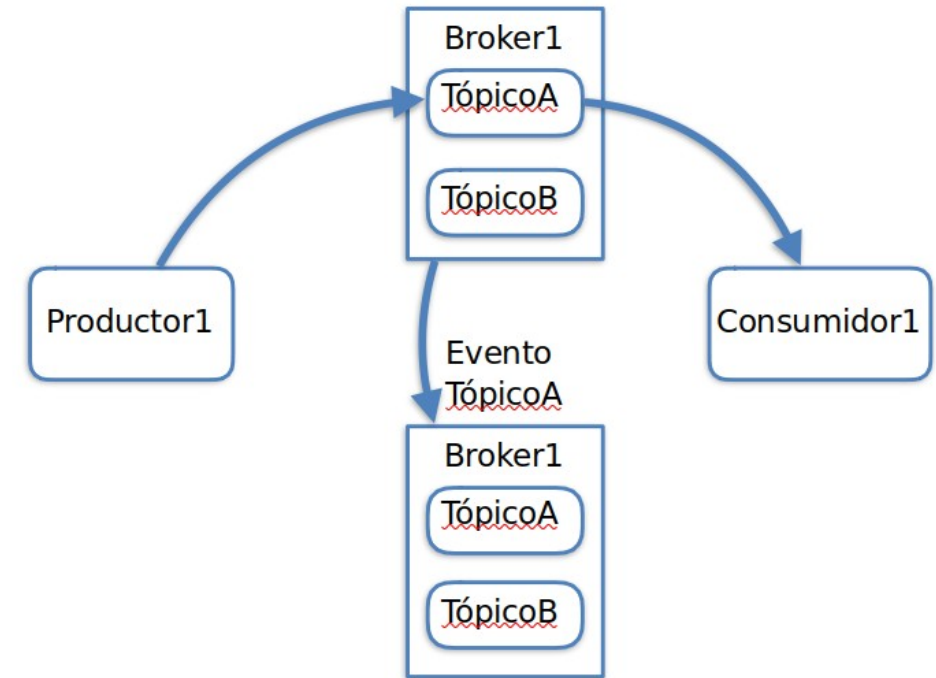
- Herramienta escalable de Streaming que actúa como un “buffer” de datos entre otras aplicaciones de Streaming
- Permite solventar el problema de backpressure para sistemas que no tienen soporte para ello
- Emisor = Productor. Receptor = Consumidores
- Se emiten y se consumen eventos de un “tópico”



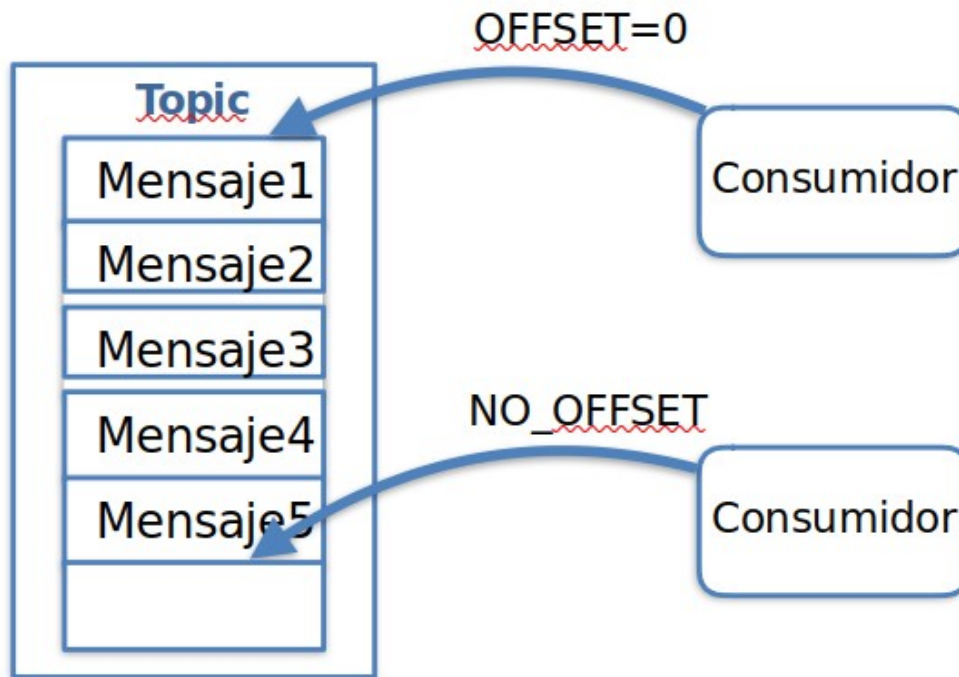
- Modelo Publish-Subscribe
- Un productor solo emite a un tópico
- Un consumidor solo recibe de un tópico
- Varios productores pueden emitir al mismo tópico
- Varios consumidores pueden recibir del mismo tópico



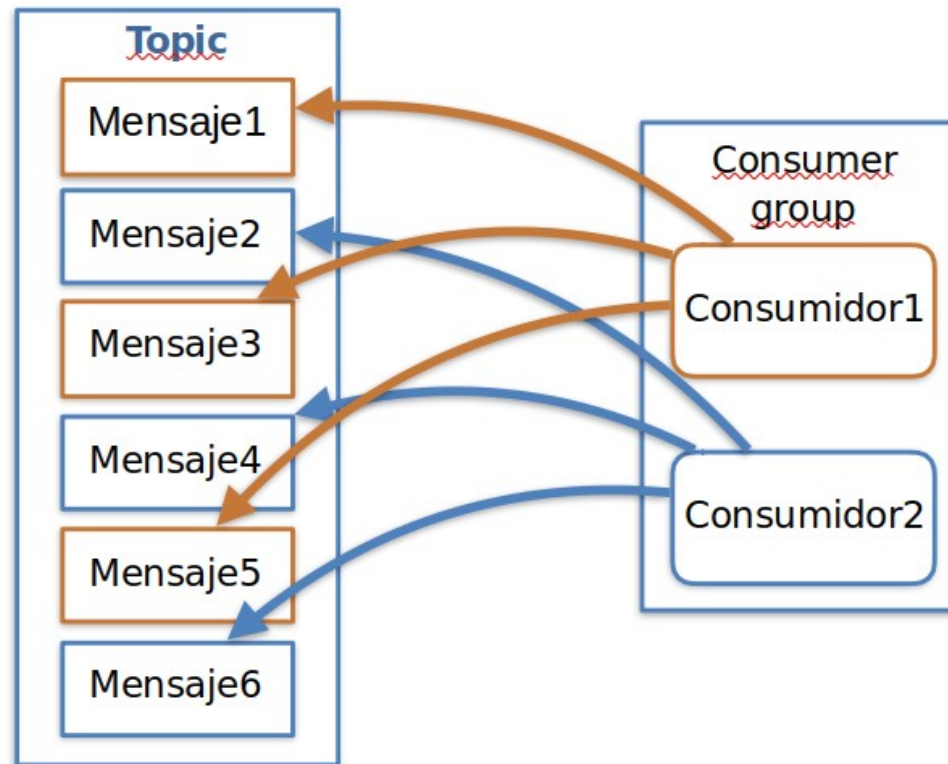
- Un broker es un servidor que permite almacenar eventos de uno o varios tópicos
- Una conjunto de brokers forman un cluster
- Cada broker contiene réplicas de datos = alta disponibilidad



- Kafka guarda todos los eventos de forma permanente (normalmente entre 2 y 7 días)
- Se puede acceder a cualquier evento a través del offset (desplazamiento)
- Por defecto, se usa el offset actual

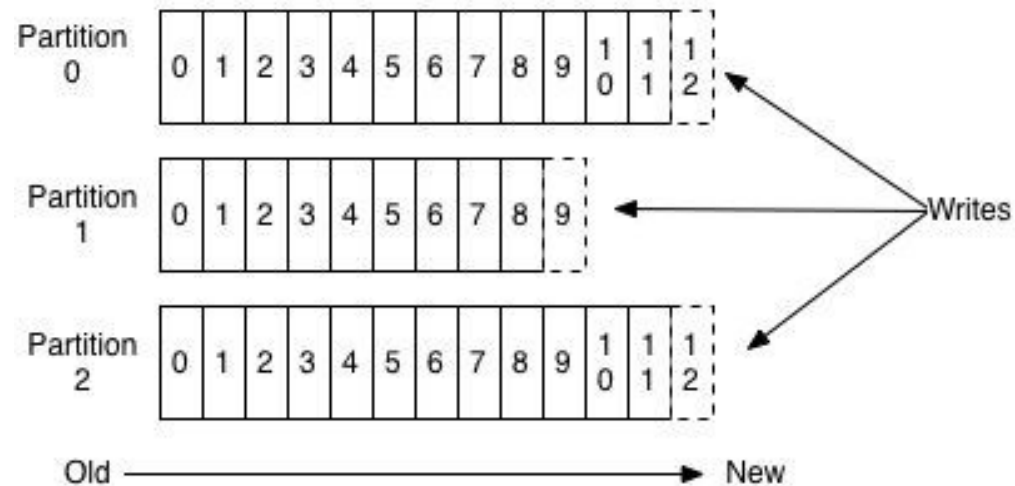


- Varios consumidores pueden compartir el offset sobre un topic, formando así un consumer group
- Cada consumidor recibe un mensaje del tópicó sin que se repita entre consumidores

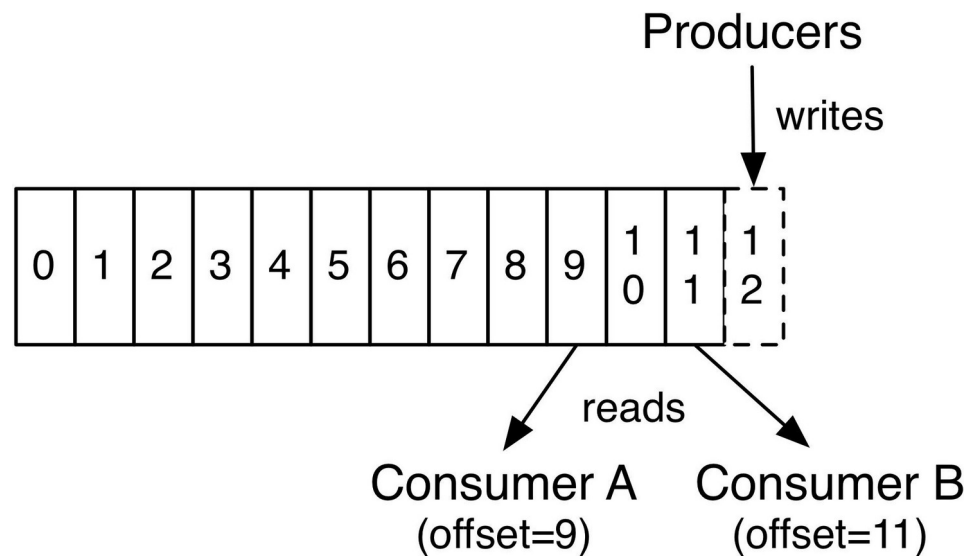


- Tópicos
  - Nombre o destino al que se publican los mensajes
  - Pueden tener 0,1, o varios consumidores
  - Se dividen en 1 o varias particiones

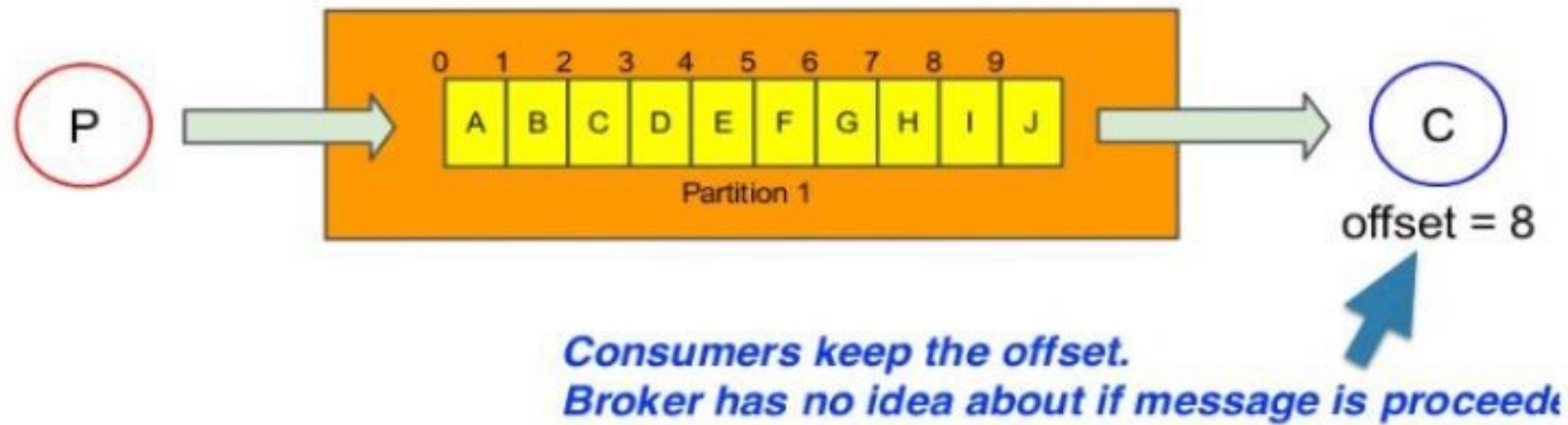
## Anatomy of a Topic



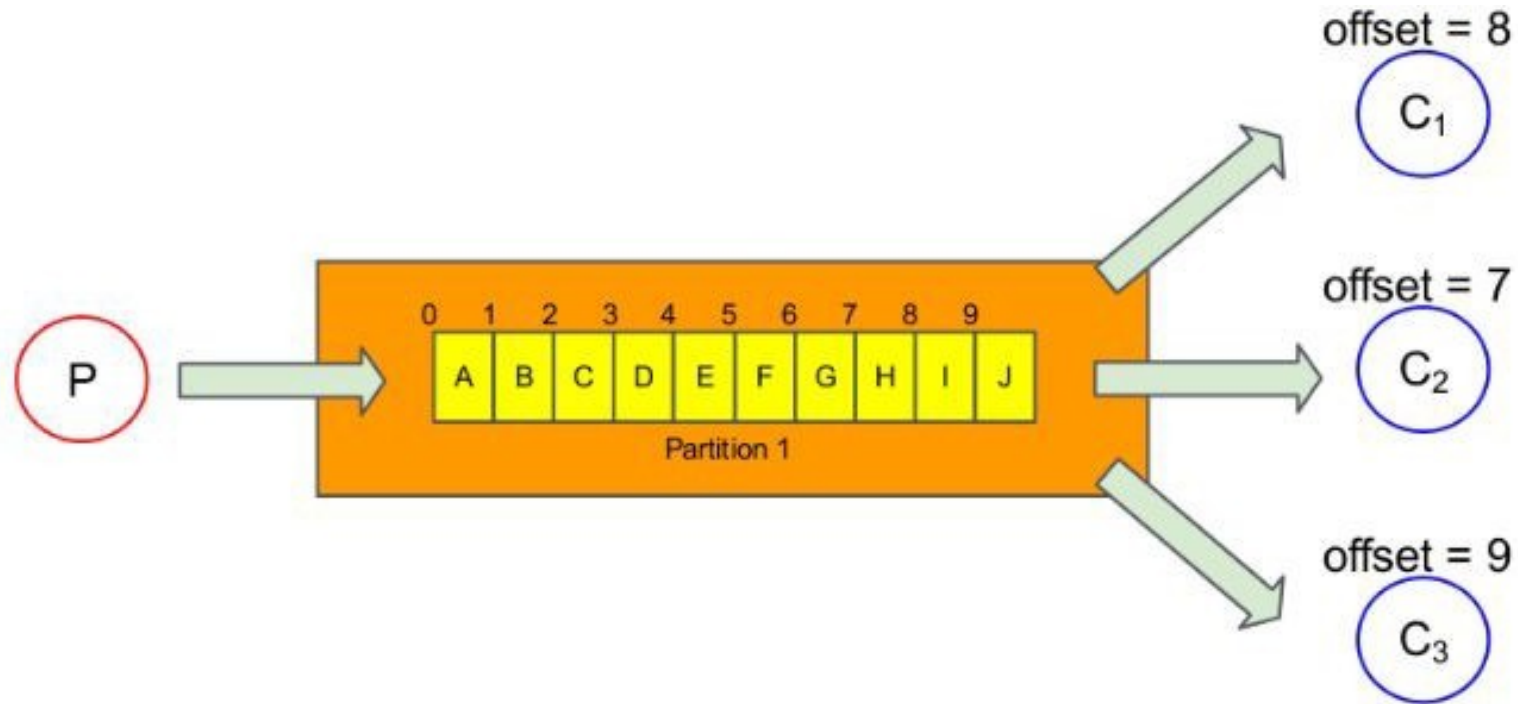
- Particiones
  - Secuencia ordenada e inmutable de registros
  - Append only (log)
  - Offset secuencial
  - Unidad de paralelismo y escalado
  - Se pueden replicar



- Modelos de consumición
  - 1 partición, 1 consumidor ==> cola

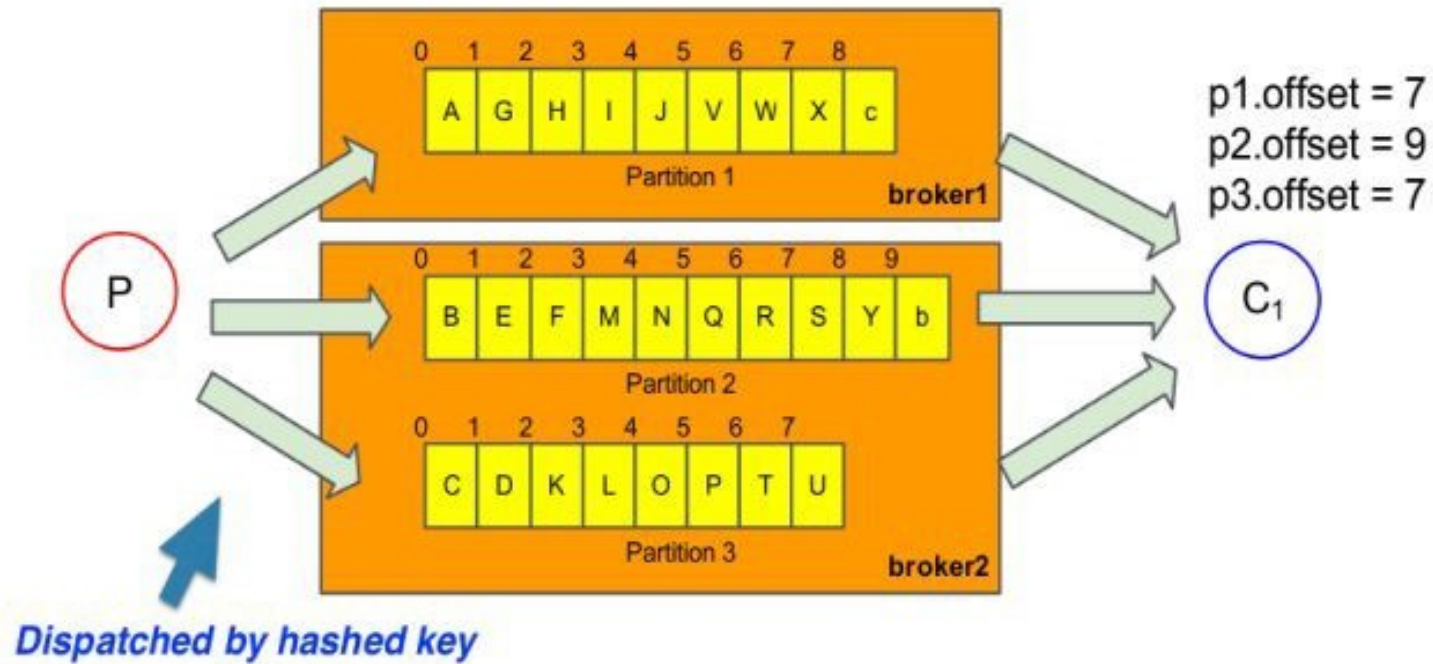


- Modelos de consumición
  - 1 partición, n consumidores ==> pub/sub

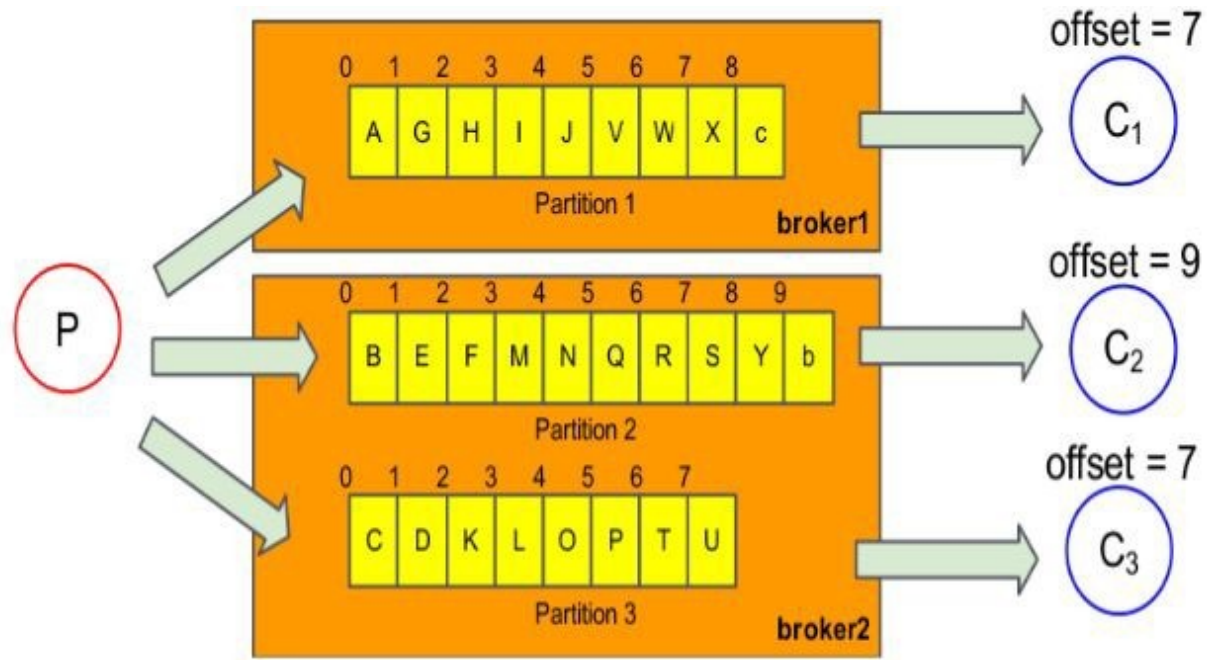


*Each Consumer keep its own offset.*

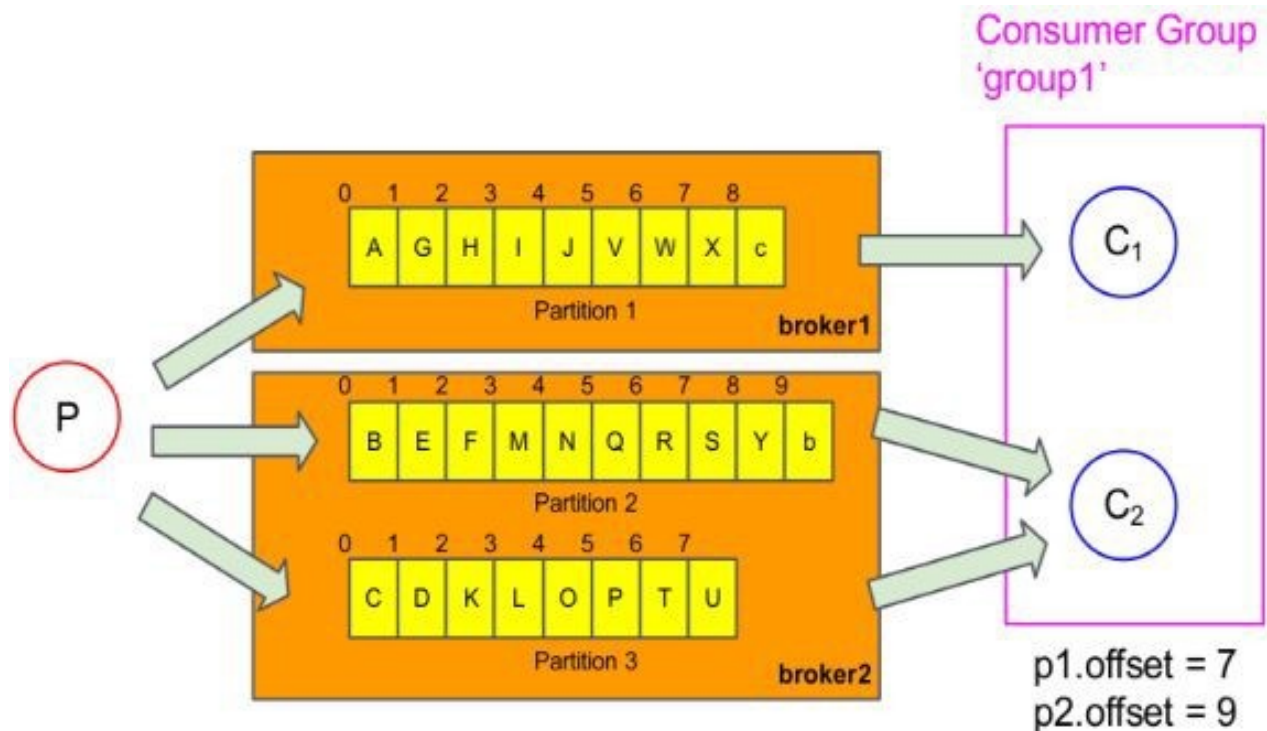
- Modelos de consumición
  - N particiones, 1 consumidor



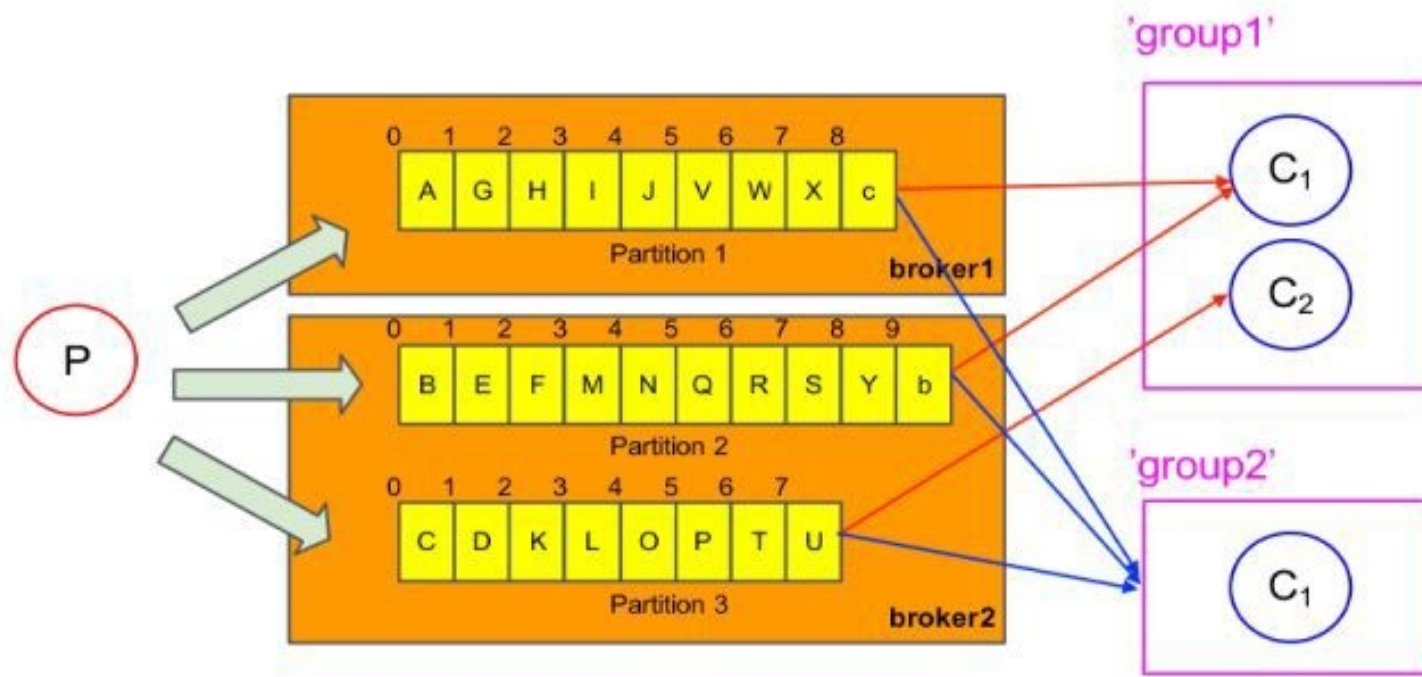
- Modelos de consumición
  - N particiones,  $n^o$  consumidores



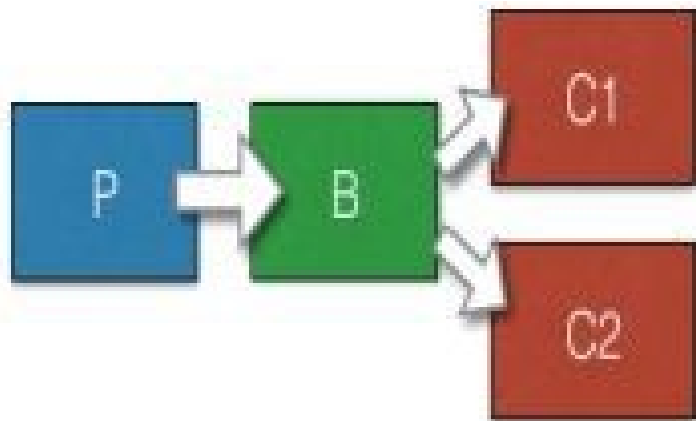
- Consumer groups
  - N particiones, 1 grupo de consumidores



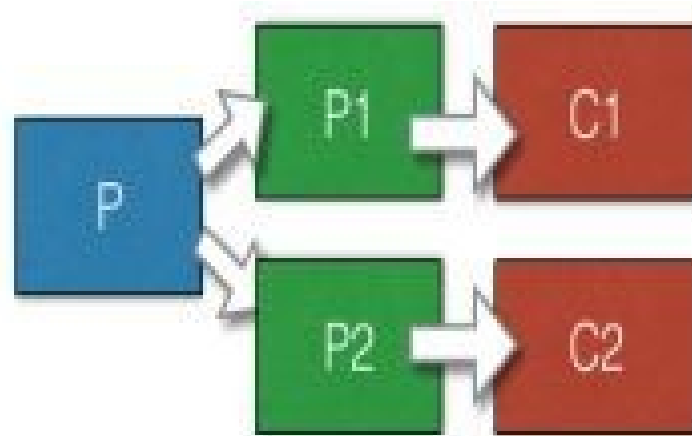
- Consumer groups
  - N particiones, 2 grupos de consumidores



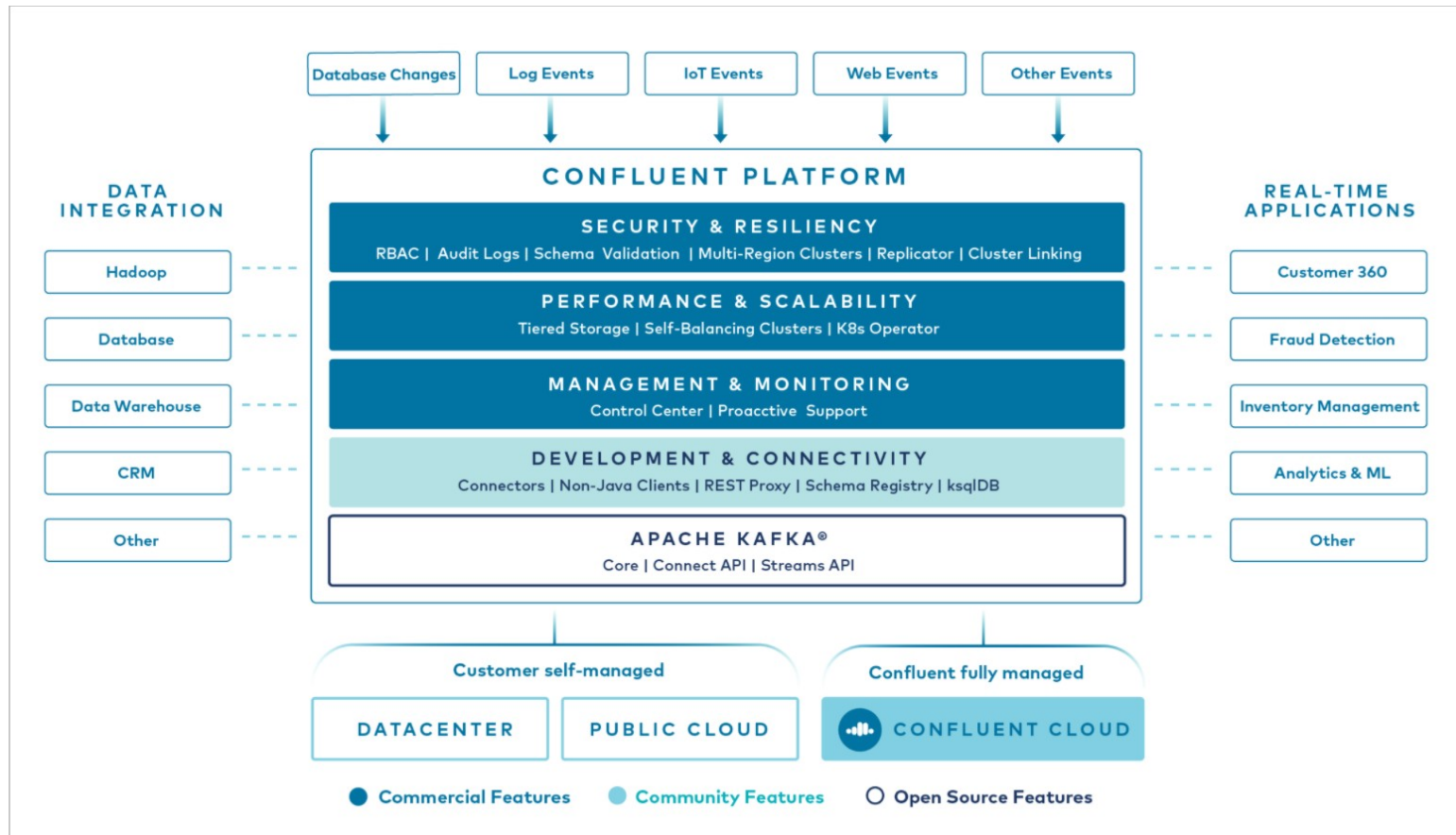
- Sistema de mensajería tradicional garantizan el orden de entrega
- Kafka garantiza el orden de procesamiento dentro de la misma partición



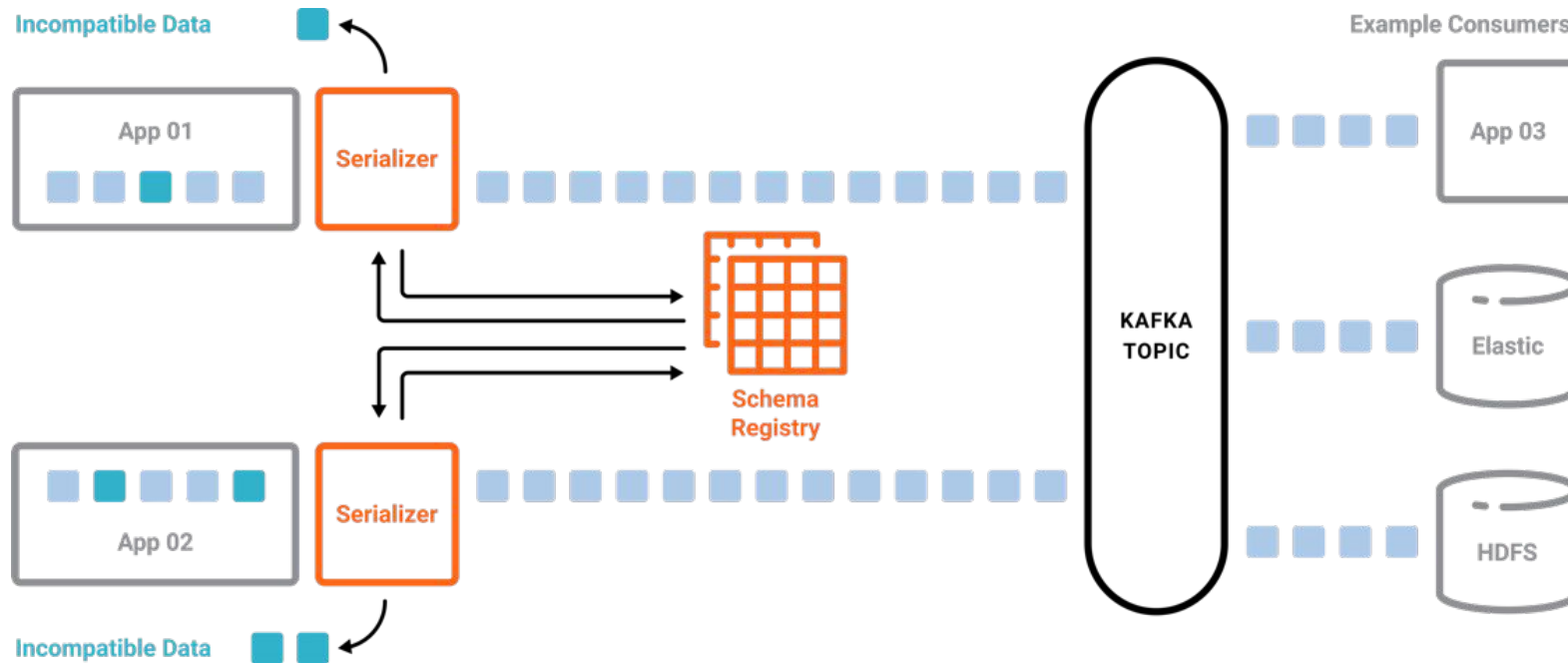
*Traditional MQ*



*Kafka*



# Schema Registry



- Repositorio de esquemas Avro
- Backend en tópicos de Kafka
- Esquemas disponibles para ser usados como clave o como valor de los mensajes y ser asociados a un tópico
- Aplicaciones cliente pueden registrar sus propios esquemas (desaconsejado en producción)
- Soporta evolución de los esquemas
- Garantiza que la evolución de esquema respeta el contrato productor-consumidor

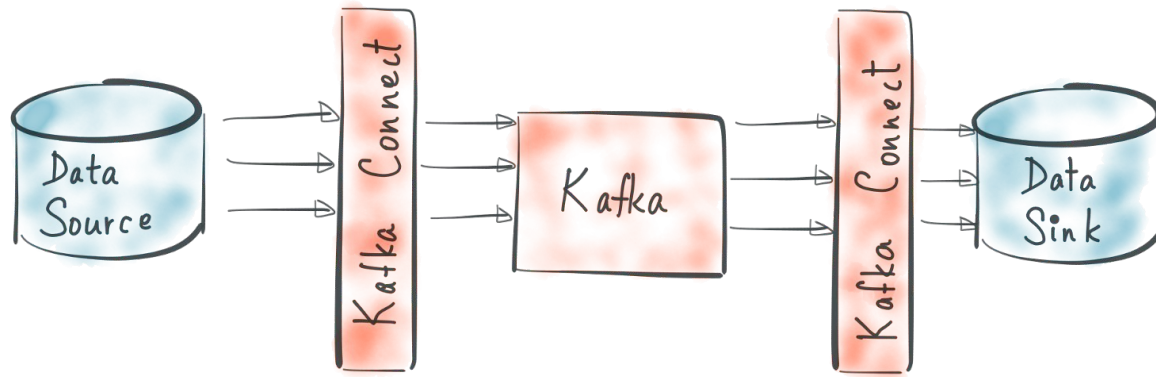
```
{
  "namespace": "io.confluent.examples.clients.basicavro",
  "type": "record",
  "name": "Payment",
  "fields": [
    {
      "name": "id",
      "type": "string"
    },
    {
      "name": "amount",
      "type": "double"
    }
  ]
}
```

● V1

```
{
  "namespace": "io.confluent.examples.clients.basicavro",
  "type": "record",
  "name": "Payment",
  "fields": [
    {
      "name": "id",
      "type": "string"
    },
    {
      "name": "amount",
      "type": "double"
    },
    {
      "name": "region",
      "type": "string",
      "default": ""
    }
  ]
}
```

● V2

## KAFKA CONNECT



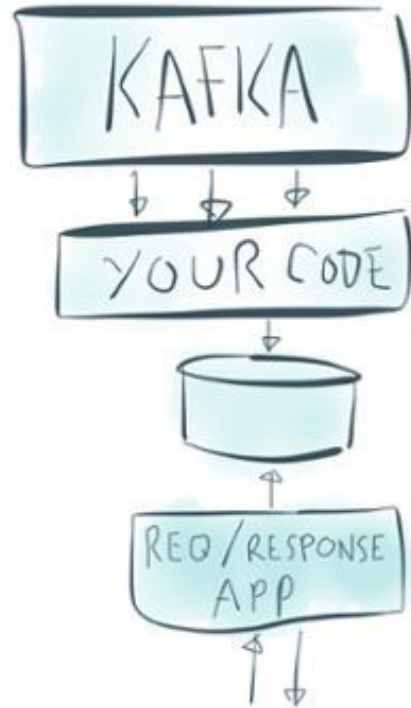
- Kafka Connect es un componente que permite leer y escribir desde/en diversas fuentes de datos, tanto de streaming como de batch



- Framework para conectar Kafka con sistemas externos
- Basado en conectores:
  - source connector (hacia Kafka)
  - sink connector (desde Kafka)
- Puede funcionar como un proceso standalone o como servicio distribuido (YARN, Mesos, Kubernetes...), escalable y tolerante a fallos
- Existen conectores soportados o podemos desarrollar nuestro propio conector

- Connectores soportados:
  - FileStream
  - HDFS
  - S3
  - Google Cloud Storage
  - JDBC
  - Elasticsearch
  - Cassandra
  - JMS
  - Active MQ IBM MQ

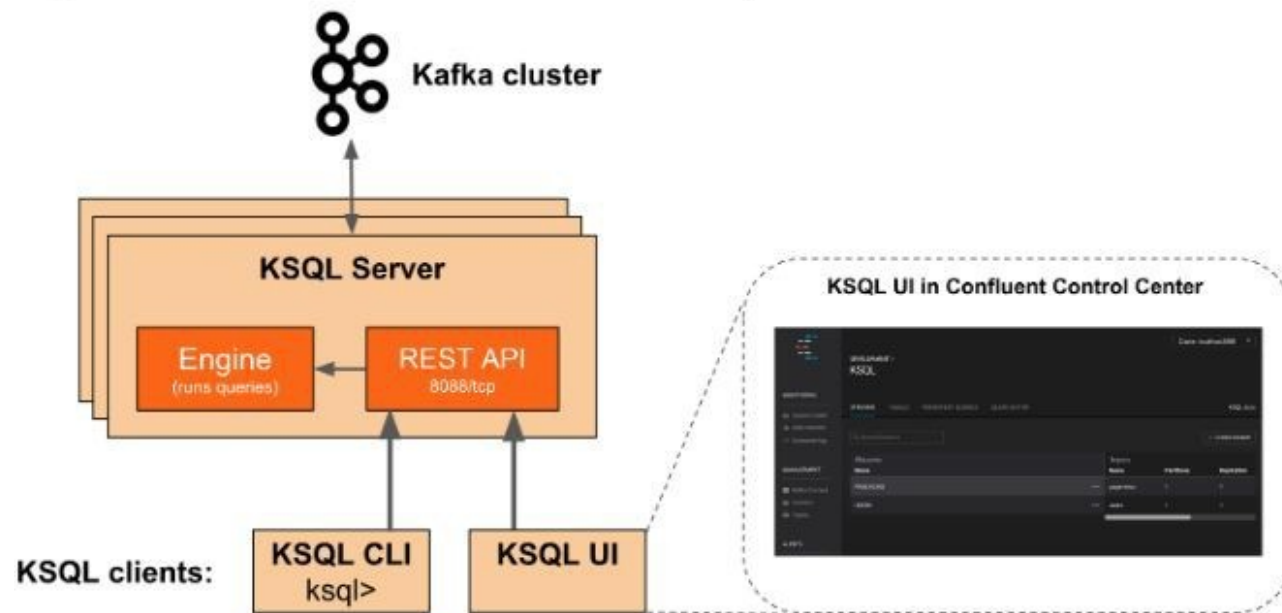
WITH KAFKA STREAMS



- Kafka Streams es un componente que permite realizar ETLs desde un topic a otro

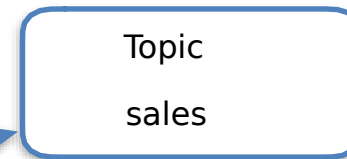
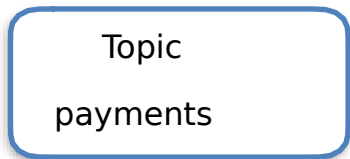


- Características:
  - Procesado de eventos de 1 en 1 (con latencias de milisegundos)
  - Operaciones con estado: agregaciones, joins, ventanas...
  - Procesamiento distribuido, tolerante a fallos
  - Fail fast
  - Capacidades de reprocesado
  - Rolling deployments sin downtime
  - API de 2 niveles



- KSQL es un componente que permite hacer lo mismo que Kafka Streams pero con sentencias SQL-like

```
CREATE STREAM sales AS SELECT  
user, sum(ventas) FROM payments GROUP BY user  
WHERE  
num_productos > 10
```

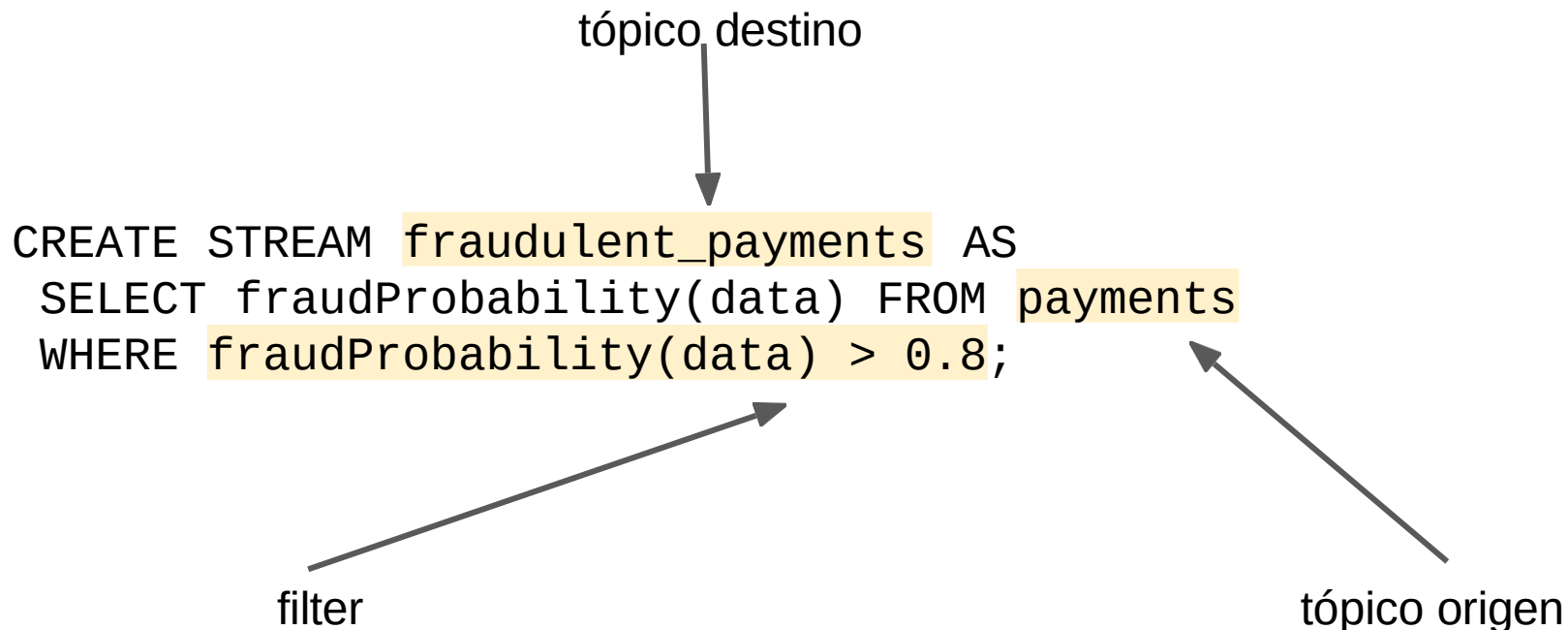


# Ksql vs Streams

```
val fraudulentPayments: KStream[String, Payment] = builder
    .stream[String, Payment]("payments-kafka-topic")
    .filter((_ , payment) => payment.fraudProbability > 0.8)

fraudulentPayments.to("fraudulent-payments-topic")
```

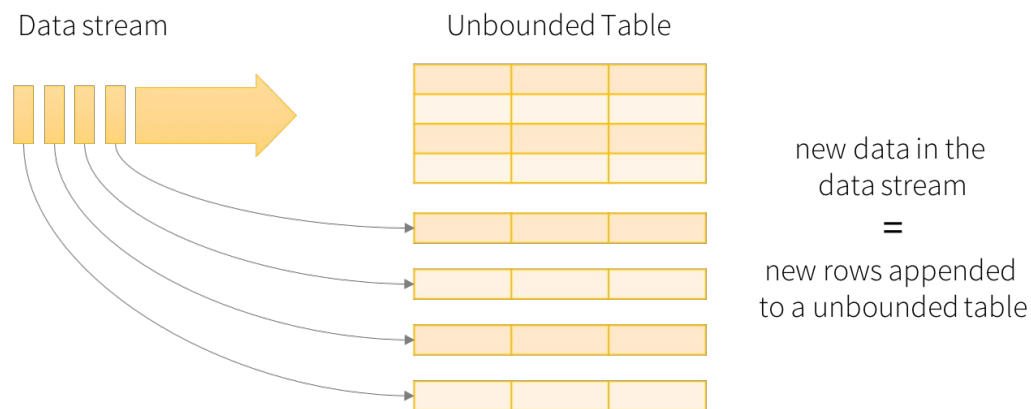
```
CREATE STREAM payments_stream ( id VARCHAR, payment STRUCT<amount DECIMAL, fraudProbability
    DECIMAL> ) WITH (KAFKA_TOPIC='payments-kafka-topic', VALUE_FORMAT='JSON');
```



# **3. Spark Streaming: Structured Streaming**

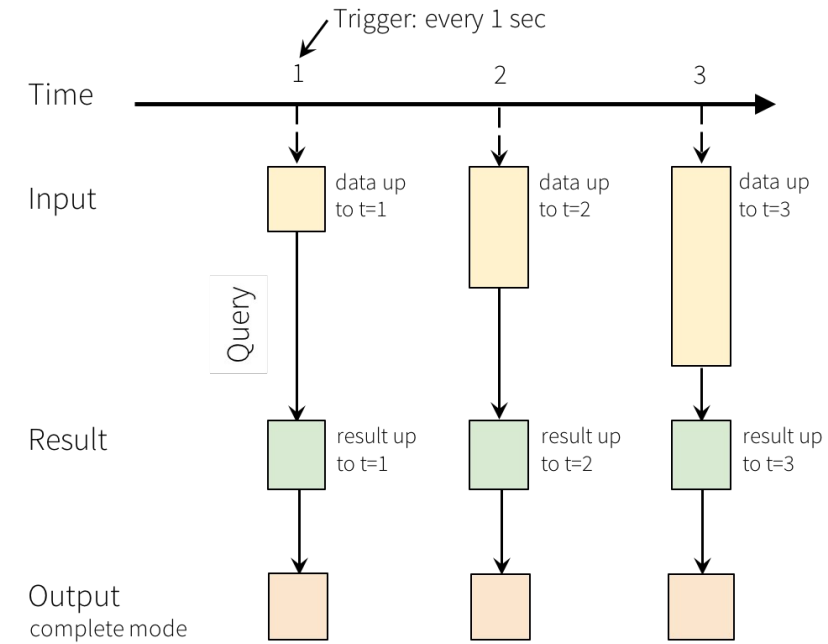
# Structured Streaming: Intro

- Motor de procesamiento de Streaming escalable y tolerante a fallos construido sobre el motor Spark SQL
- La lógica de computación de Streaming es similar a la de la computación estática en Batch
- Spark SQL se encarga de ejecutar de forma incremental y continua, y actualizar el resultado final a medida que los datos de Streaming siguen llegando
- Se utiliza la misma API de DataFrame API para expresar agregaciones streaming, ventanas de tiempo de eventos, uniones de datos en Streaming con datos batch
- Garantiza la tolerancia a fallos de punto a punto y entrega Exactly-Once a través de checkpoints
- Por defecto, las queries son procesadas como una serie de micro-batches, con latencias de 100 ms
- La idea es tratar los datos Streaming como una tabla a la que se añaden datos continuamente
- Desarrolla como una query en Batch sobre datos estáticos y Spark ejecutará esa lógica incrementalmente sobre datos infinitos



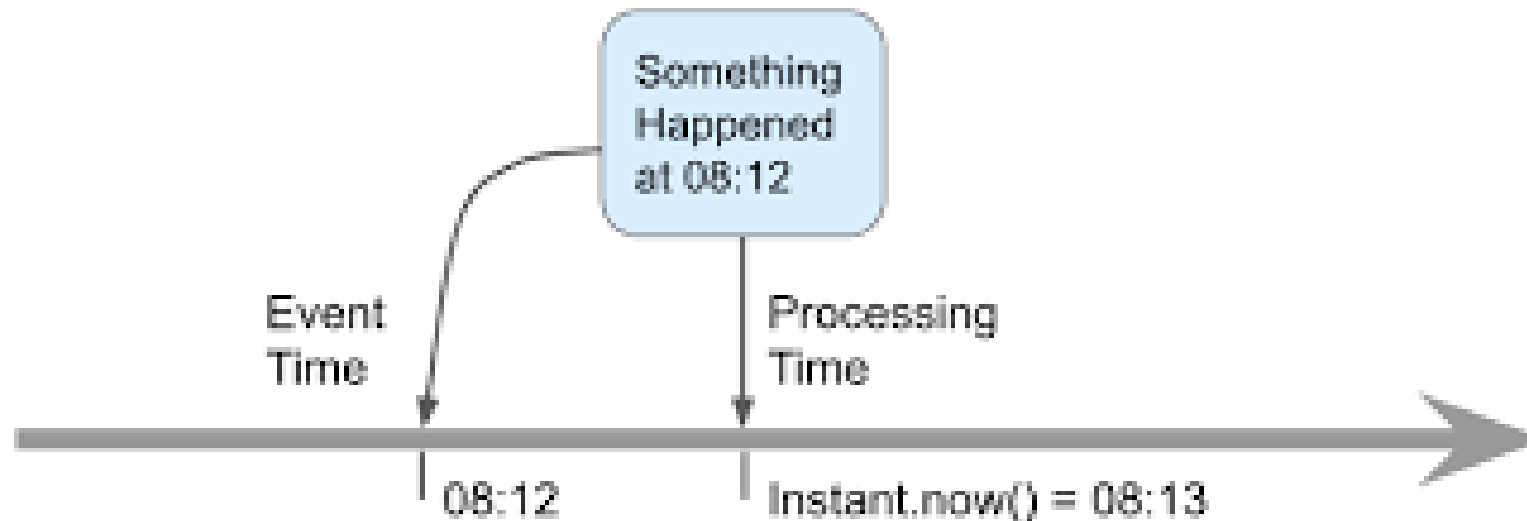
Data stream as an unbounded table

- Una query sobre los datos entrantes genera una Result Table
- En cada intervalo, se añaden nuevas filas a la Input Table, lo que hace que se actualice la Result Table
- Cada vez que se actualiza el resultado, este resultado se puede escribir en una fuente externa (Sink)
- El resultado puede ser escrito en diferentes modos:
  - Complete: Todos los datos en la Result Table se escriben en el almacenamiento externo
  - Append - Solo las filas nuevas se escriben en el almacenamiento externo. Solo aplicable a queries que no afectan a filas ya existentes en la Result Table
  - Update - Solo escribe aquellas filas que se actualizaron en la Result Table
- En la práctica, no se materializa toda la tabla entera
- El motor recibe los últimos datos de la fuente de Streaming, los procesa para actualizar el resultado y descarta los datos de origen
- Tan solo mantiene los datos mínimos del estado que son requeridos para actualizar el resultado



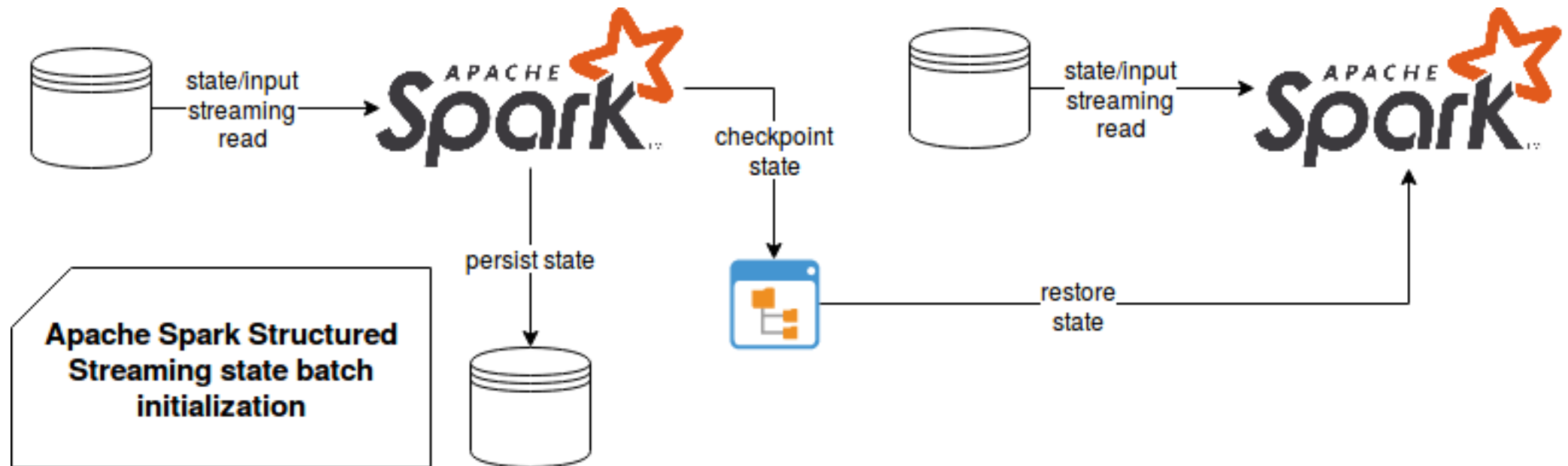
Programming Model for Structured Streaming

- El Event-time es el tiempo embebido en los propios datos
- Cada evento se representa como una fila en la tabla y event-time es el valor de una columna en cada fila
- Cada ventana de tiempo es una agrupación de filas y cada fila puede pertenecer a varias ventanas
- Este modelo permite tratar datos que llegan más tarde del cierre de ventana
- El motor puede actualizar los datos de la Result Table con datos que llegan tarde ya que guarda datos intermedios de ventanas anteriores
- Las marcas de agua permiten al usuario especificar el umbral de espera de datos tardíos y así que el motor puede realizar una limpieza de los datos intermedios de ventanas anteriores



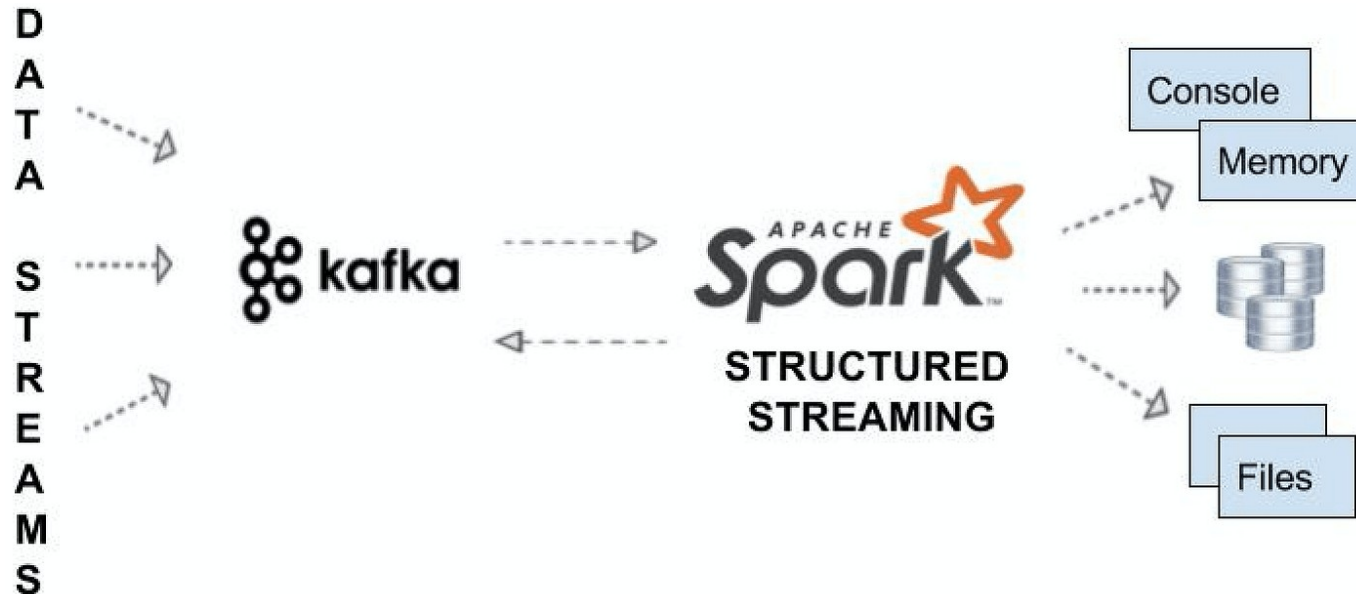
# Structured Streaming: Fault-Tolerance

- El motor monitoriza constantemente el progreso del procesamiento para poder tratar cualquier error reiniciando o reprocesando los datos necesarios
- Para ello, se asume que las fuentes Streaming utilizan conceptos como Offsets o posiciones de lectura en el Stream
- El motor utiliza checkpoints y logs de escritura anticipada para registrar el offset de los datos que se están procesando en cada ventana.
- Los Sinks están diseñados para ser idempotentes y así soportar el reprocesamiento
- Spark garantiza una entrega Exactly-Once de extremo a extremo bajo cualquier fallo



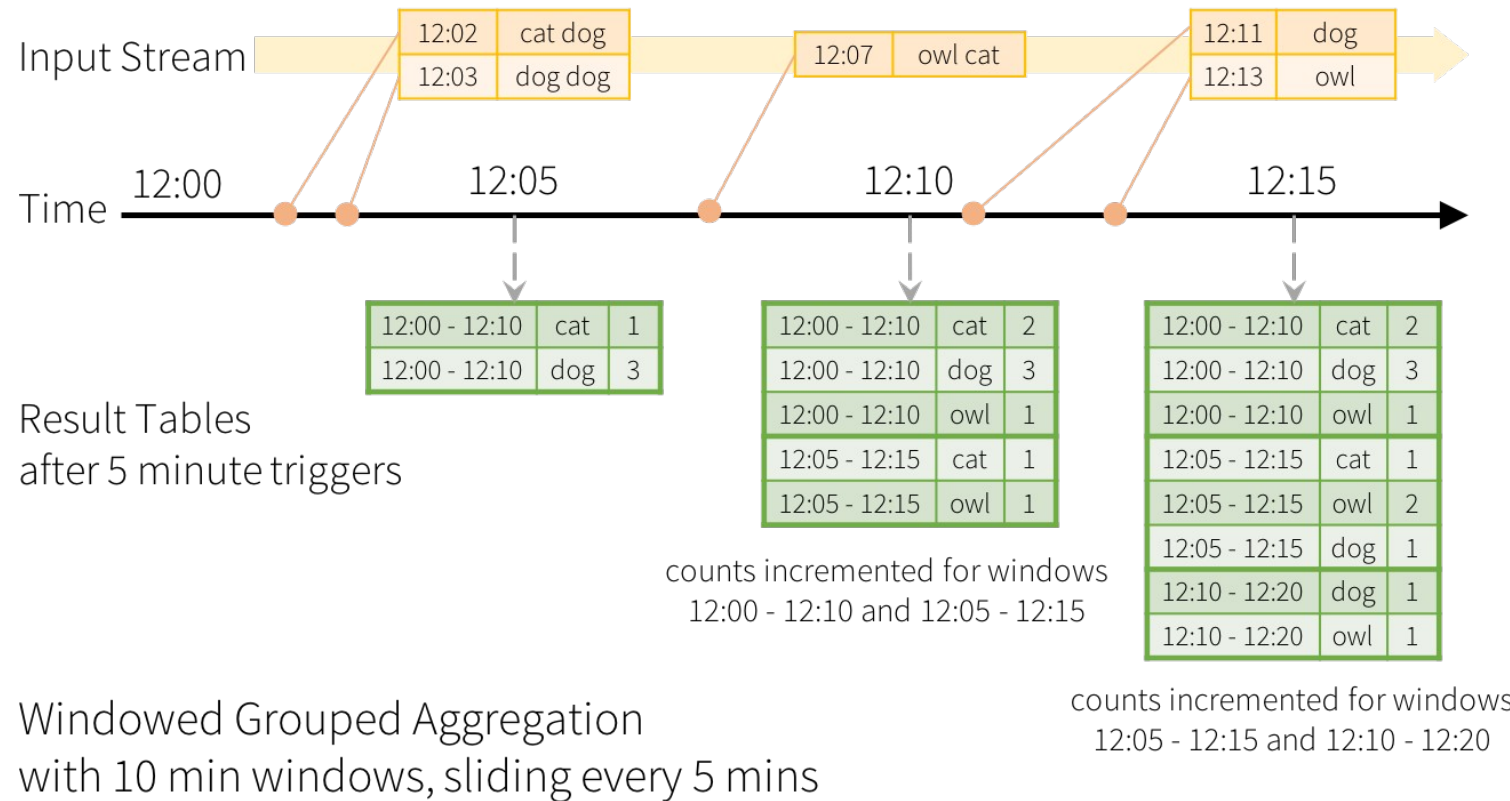
# Structured Streaming: Input Sources

- Los DataFrames pueden representar datos estáticos y acotados, así como datos infinitos de Streaming
- Similar a los Datasets/DataFrames estáticos, SparkSession es el punto de entrada para crear DataFrames de Streaming a partir de fuentes de streaming y aplicar las mismas operaciones en ellos que en los DataFrames estáticos
- SparkSession.readStream() → [DataStreamReader](#)
- Cada fuente se define a través de una serie de propiedades como formato, esquema, opciones, etc
  - File - Lee archivos escritos en un directorio como un flujo de datos. Los archivos se procesan por orden de modificación. Si latestFirst está configurado, el orden se invertirá. Los formatos de archivo admitidos son texto, CSV, JSON, ORC, Parquet.
  - Kafka - Lee datos de [Kafka](#).
  - Socket - Lee datos de texto de una conexión Socket.
  - Rate - Genera datos a la velocidad especificada de filas por segundo, cada fila de salida contiene una marca de tiempo y un valor. El timestamp contiene la hora de envío del mensaje, y un valor tipo Long que contiene el recuento de mensajes, comenzando desde 0 como la primera fila
  - Rate Per Micro-Batch - Igual que Rate pero los datos se acumulan en un micro-batch



# Spark Streaming: Operations

- Básicas
  - La mayoría de las operaciones comunes en DataFrame/Dataset son compatibles para Streaming.
  - Se puede registrar un DataFrame de Streaming como una vista temporal para luego realizar consultas SQL
- Window Operations on Event Time
  - Los valores agregados se mantienen para cada ventana que se produce

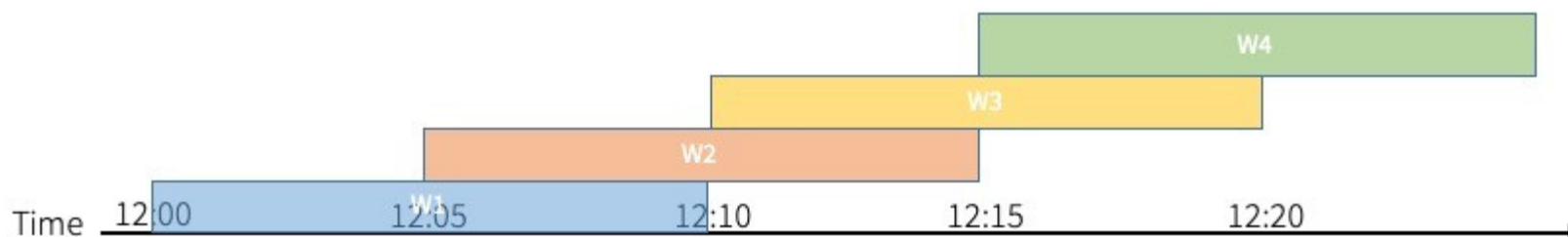


- Tipos de ventanas

Tumbling Windows (5 mins)



Sliding Windows (10 mins, slide 5 mins)

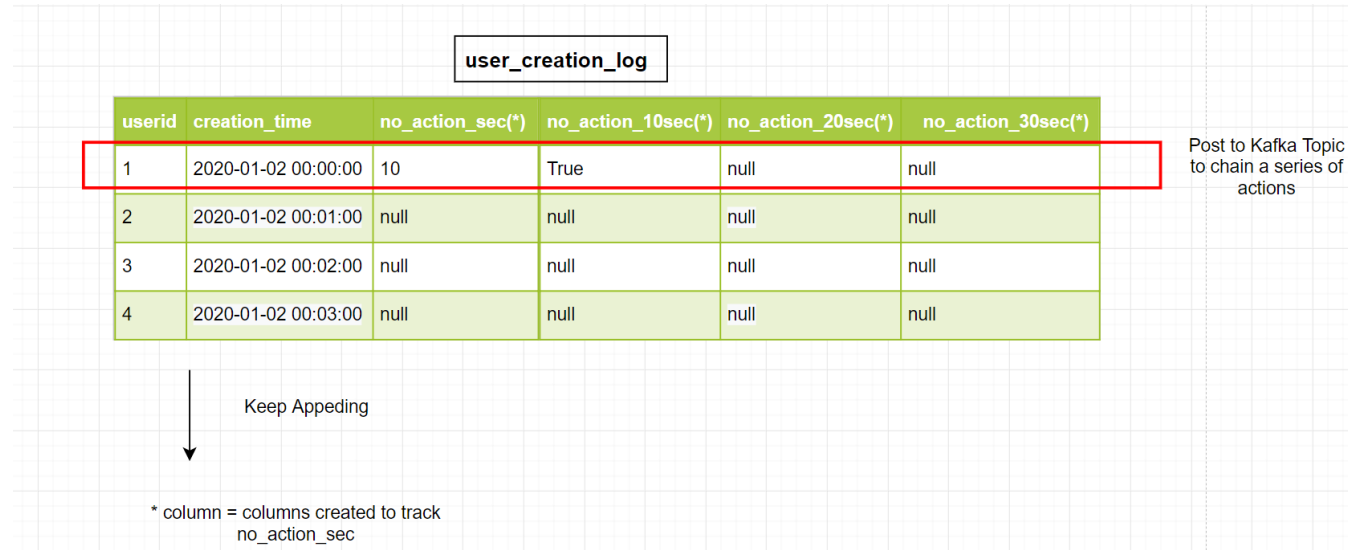


Session Windows (gap duration 5 mins)



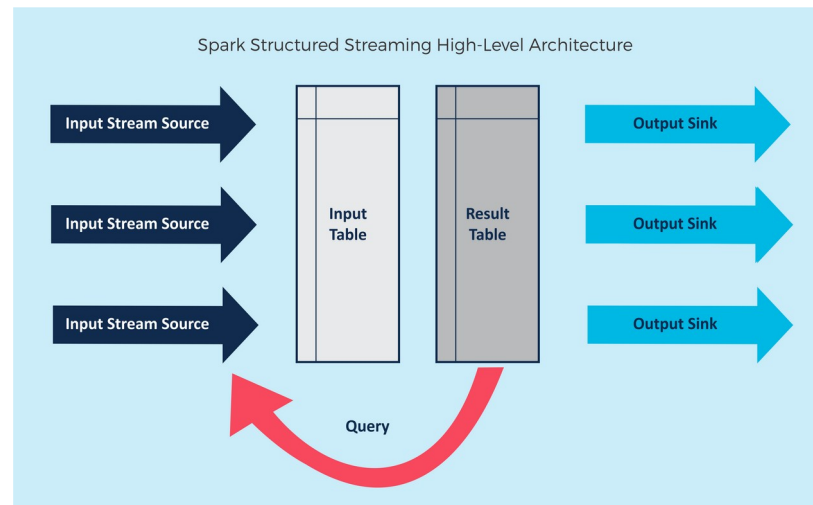
# Spark Streaming: Output Sinks

- Para iniciar el proceso Streaming, se debe usar el DataStreamWriter devuelto de Dataframe.writeStream()
  - Detalles del Sink destino: Formato de datos, ubicación, etc.
  - Modo de salida: Especifica qué se escribe en el destino de salida.
    - Append (default) - Solo se escribirán en el destino las nuevas filas agregadas a la Result Tabla desde el último trigger de ventana.
    - Complete - Toda la Result Tabla se escribirá en el destino después de cada trigger.
    - Update - Solo se escribirán en el Sink las filas de la Result Tabla que se hayan actualizado desde el último trigger.
  - Query name: Opcionalmente, especifica un nombre único de la consulta para su identificación
  - Trigger interval: Opcional. Si no se especifica, el sistema verificará la disponibilidad de nuevos datos tan pronto como se haya completado el procesamiento anterior.
  - Checkpoint location: Para algunos Sinks donde se puede garantizar la tolerancia a fallos de extremo a extremo, especifica la ubicación donde el sistema escribirá toda la información de checkpoint. Debe ser un directorio en un sistema de archivos tolerante a fallos compatible con HDFS.



# Spark Streaming: Output Sinks

- Sinks
  - File - Almacena la salida en un directorio
  - Kafka - Almacena la salida en uno o más temas en Kafka
  - Foreach - Ejecuta cálculos arbitrarios en los registros de salida
  - ForeachBatch - Reutiliza fuentes de datos Batch existentes y escribe en múltiples ubicaciones
  - Console - Imprime la salida en la consola/stdout cada vez que se activa
  - Memory - La salida se almacena en la memoria como una tabla en memoria
- Streaming Table
  - Se puede usar `DataStreamReader.table()` para leer tablas como DataFrames de streaming y usar `DataStreamWriter.toTable()` para escribir DataFrames de Streaming como tablas
- Triggers
  - Default: ejecuta micro-batches tan pronto como sea posible
  - ProcessingTime: Intervalo micro-batch
  - Once: la consulta ejecutará solo un micro-batch para procesar todos los datos disponibles y luego se detendrá
  - AvailableNow: la consulta procesará todos los datos disponibles y luego se detendrá



- Streams
  - `spark.streams.active` // lista de consultas de streaming actualmente activas
  - `spark.streams.get(id)` // query por su identificador único
  - `spark.streams.awaitAnyTermination()` // bloquearse hasta que una de ellas finalice
- Management
  - `query.id` // identificador único de la query en ejecución que persiste
  - `query.runId` // id único de la ejecución en curso de la query
  - `query.name` // nombre auto-generado o especificado por el usuario
  - `query.explain()` // imprime explicaciones detalladas de la query
  - `query.stop()` // detiene la query
  - `query.awaitTermination()` // se bloquea hasta que la query termine con `stop()` o con error
  - `query.exception` // excepción si finalizó con error
  - `query.recentProgress` // un array de los progresos más recientes para esta consulta
  - `query.lastProgress` // actualización de progreso más reciente de esta query Streaming
- Monitoring
  - `query.status` // message, `isDataAvailable`, `isTriggerActive`



 Calle Playa de Liencres, 2 bis  
(entrada por calle Rozabella)  
Parque Europa Empresarial  
Edificio Madrid  
28290 Las Rozas, Madrid

 900 373 379  [info@u-tad.com](mailto:info@u-tad.com)

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:

 **Universidad  
Camilo José Cela**

PROYECTO COFINANCIADO POR:

